

TRANSITION AND RECOVERY ALIGNMENT VIA EVIDENCE-GROUNDED CONTRACTS FOR MOBILE GUI AGENTS

Xingjian Diao

Dartmouth College
NH, USA

ABSTRACT

Mobile GUI agents execute natural-language instructions through sequences of interface actions, but long-horizon execution remains vulnerable to silent transition failures, state drift, and ineffective recovery. A plausible action may fail to produce the intended state transition, causing subsequent decisions to be conditioned on a state that was never reached and allowing these errors to compound over time. Existing systems often address such failures through online reflection or auxiliary verification, introducing additional reasoning and model calls during execution. We introduce TRAVEC, a framework that moves transition prediction, verification, and recovery from the deployment-time execution loop into training. The key abstraction is an evidence-grounded *transition contract*, which specifies the task-relevant state change expected after an action, the observable evidence used to verify that change, and an optional recovery conditioned on transition failure. From heterogeneous mobile GUI trajectories, we construct 550K verifiable supervision items and use them to train a Contract Policy and a Verifier that audit self-generated trajectories and retain reliable training steps. The resulting audited experience is then used to optimize the Action Policy through supervised learning and offline GRPO. These auxiliary components are required only during training; deployment retains a single action-only policy with no additional verification or free-form reasoning. Across six offline and online benchmarks, TRAVEC improves grounding and sequential execution while retaining low inference latency, indicating that transition-aware verification and recovery signals can be incorporated into policy learning without adding deployment-time reasoning overhead.

1 INTRODUCTION

Graphical user interfaces (GUIs) are the primary means through which people access mobile applications, desktop software, and web services, making them a natural environment for agents that translate natural-language instructions into digital actions (Wang et al., 2024b; Tang et al., 2025). In particular, mobile GUI agents execute multi-step tasks through repeated cycles of interface perception, planning, and action generation (Tang et al., 2025; Zhou et al., 2026a). However, a plausible action is not necessarily a successful one: it may be correctly formed and targeted yet fail to induce the intended state transition because of overlays, focus shifts, or delayed interface updates. If the agent proceeds as though the action succeeded, subsequent decisions are conditioned on a state that was never reached. Figure 1 illustrates three representative failure modes in long-horizon mobile GUI interac-

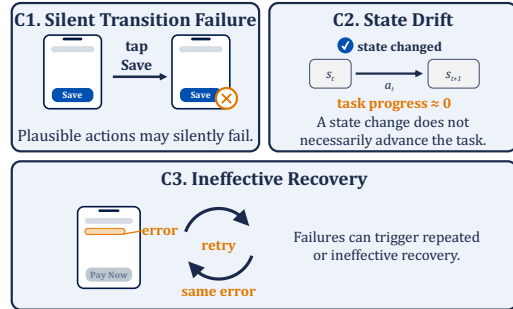


Figure 1: Representative failure modes in long-horizon mobile GUI interaction: (C1) plausible actions can fail silently, (C2) state changes may not indicate task progress, and (C3) recovery attempts may fail to resolve execution errors.

tion: silent transition failures, state drift despite observable changes, and ineffective recovery. Over extended trajectories, such local failures can accumulate and ultimately lead to task failure (Rawles et al., 2025; Yu et al., 2026).

Existing GUI-agent supervision is largely organized at two levels. Action-level grounding and imitation data teach a policy what to do and where to act (Hong et al., 2024; Cheng et al., 2024), while trajectory-level environments provide task-level signals of overall task completion (Xie et al., 2024; Rawles et al., 2025; Lai et al., 2026; Liu et al., 2026b). What remains less explicit is the local transition between these two levels: given the current interface and an action, what task-relevant change should occur next, and what evidence would verify that change? Without such a target, a policy may observe either no state change or an unintended one and still continue as if the action had succeeded.

Recent work addresses action consequences through state-transition or world-model pretraining (Liu et al., 2026c; Li et al., 2025b), process rewards and visual state comparison (Chen et al., 2025; Qian, 2026), and state-aware reflection or recovery (Guo et al., 2026b; Zhang et al., 2026; Guo et al., 2026a). However, transition prediction, outcome verification, and recovery are often developed as separate components and, in many systems, remain part of the execution loop, where additional reasoning and model calls increase per-step latency (Huang et al., 2026). This raises a natural question: can these capabilities be unified as training-time supervision and removed from deployment?

We answer this question with an evidence-grounded *transition contract*. Rather than predicting the full next screen, a contract captures only the task-relevant consequence of an action: the current state and preconditions, expected state change, observable evidence, and an optional recovery. Post-action observations serve only as privileged training information for contract construction and validation; they never enter the Action Policy’s decision-time input. The contract therefore provides a unified training interface for transition prediction, outcome verification, and failure-conditioned recovery.

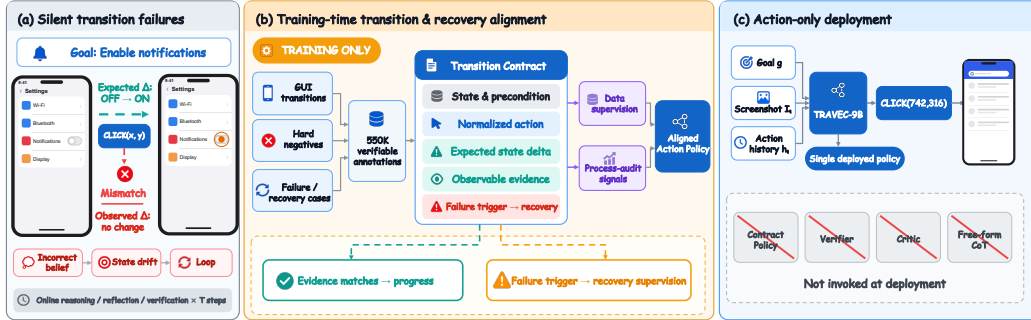


Figure 2: Overview of TRAVEC. Transition prediction, outcome verification, and failure recovery are incorporated into training through evidence-grounded contracts and process-audit signals. Deployment retains only a single action-only policy, with all auxiliary components removed.

Building on this representation, we introduce TRAVEC, a transition-aware training framework for mobile GUI agents, illustrated in Figure 2. We construct 550K verifiable supervision items using deterministic state differencing, evidence-grounded multimodal annotation, counterfactual recovery generation, and model-independent validation. A Contract Policy predicts expected transitions before execution, while a Verifier evaluates realized outcomes afterward; together with a trajectory Critic, they audit self-generated interactions and curate them into training data for policy optimization. The Action Policy is then optimized through supervised learning and offline GRPO with action-alignment and audited process signals. All auxiliary components are used only during training. At deployment, TRAVEC retains a single Action Policy that emits one executable action per step without auxiliary verification or free-form reasoning. Our contributions are threefold:

- We develop a scalable automated pipeline for constructing transition-aware GUI supervision. Using evidence-grounded transition contracts, deterministic state differencing, multimodal annotation, and counterfactual recovery generation, the pipeline produces 550K verifiable supervision items capturing expected state changes, observable evidence, and failure-conditioned recovery.

- We introduce a training-time process-audit framework that combines a Contract Policy, a Verifier, and a trajectory Critic to audit self-generated interactions and curate training data for supervised learning and offline GRPO.
- We train TRAVEC as a standalone action-only policy, using transition verification and recovery only during training. Across six offline and online benchmarks, TRAVEC improves grounding and closed-loop execution while retaining low inference latency without auxiliary verification or free-form reasoning at deployment.

2 RELATED WORK

2.1 GUI GROUNDING, PLANNING, MEMORY, AND RECOVERY

Navigating ambiguous graphical interfaces necessitates robust visual grounding and sequential planning capabilities. Early methodologies primarily optimized spatial resolution and visual-token efficiency to bridge the gap between natural-language instructions and local interface elements (Hong et al., 2024; Cheng et al., 2024; You et al., 2024; Lu et al., 2024; Gou et al., 2025; Wu et al., 2025; Lin et al., 2024). Building upon this perceptual foundation, native action models emerged to support cross-platform action generation and trajectory learning (Xu et al., 2024; Qin et al., 2025; Liu et al., 2026d; Ye et al., 2025; Xu et al., 2026). To manage the compounding complexity of long-horizon tasks, contemporary architectures frequently incorporate episodic memory, semantic retrieval, and explicit System-2 reflection (Wang et al., 2024a; Li et al., 2024b; Agashe et al., 2025a;b; Qin et al., 2025; Wang et al., 2025; Zhou et al., 2026b; Liu et al., 2026a; Xiao et al., 2026; Lu et al., 2026). However, many of these orchestration mechanisms retain additional memory, reflection, or verification components during execution, which can increase per-step inference cost. TRAVEC instead emphasizes transition-aware supervision that integrates prediction, verification, and recovery within a unified training framework.

2.2 AUTOMATED EXPERIENCE, REINFORCEMENT LEARNING, AND EFFICIENT EXECUTION

The scalability of GUI agents increasingly relies on automated environment exploration and synthetic trajectory generation (Sun et al., 2025; Mu et al., 2025; Cheng et al., 2026). To align these generated experiences with successful task outcomes, recent frameworks apply rule-based reward shaping or partially verifiable Group Relative Policy Optimization offline (Luo et al., 2025; Zhang et al., 2025; Yang et al., 2026; Lu et al., 2025b). Recent systems further expand this learning paradigm by integrating dynamic task generation and post-hoc evidence auditing (Lai et al., 2026; Xu et al., 2025; Liu et al., 2026b; Li et al., 2026). Recent efforts to reduce inference latency include context compression and anticipated decoding (Huang et al., 2025; 2026; Dong et al., 2026). TRAVEC takes a complementary approach by moving transition prediction, evidence-grounded verification, and recovery supervision into the training pipeline. As a result, deployment retains a single action-only policy without auxiliary verification or free-form reasoning.

3 METHOD

TRAVEC learns transition reasoning and recovery through automated supervision and process-audited optimization, yielding an action-only policy.

3.1 PROBLEM FORMULATION

We model goal-conditioned GUI interaction as a partially observable decision process. The observation $o_t = (I_t, U_t, X_t)$ comprises a screenshot, available interface structure, and extracted text. During attempt k of task i , the Action Policy conditions on screenshot I_t , goal g , action history $h_t = (a_1, \dots, a_{t-1})$, and earlier-attempt hints $\eta_i^{(<k)}$:

$$\pi_{\theta_A}(a_t \mid g, I_t, h_t, \eta_i^{(<k)}), \quad a_t = (\alpha_t, \psi_t). \quad (1)$$

Here α_t and ψ_t denote action type and arguments. We maximize $\mathbb{E}[Y(\tau)]$ for binary task outcome $Y(\tau)$. Deployment requires one valid action per step with empty hints and no auxiliary model calls.

A transition contract specifies expected consequences and recovery:

$$c_t = (\tilde{s}_t, p_t, a_t, \hat{d}_t, E_t, \rho_t). \quad (2)$$

Its components are task state $\tilde{s}_t$, precondition p_t , action a_t , expected change $\hat{d}_t$, observable evidence E_t , and recovery specification ρ_t . Recovery binds a failure trigger to a corrective action, expected change, and evidence. The Contract Policy models $q_{\theta_C}(c_t | g, o_t, h_t, a_t)$ before execution. After execution, the Verifier models $V_{\theta_V}(y_t^V | g, o_t, c_t, o_{t+1}, d_t, \chi_t)$ using the observed difference $d_t = \mathcal{D}(o_t, o_{t+1})$ and validation indicators χ_t . The output $y_t^V = (v_t, \nu_t, \kappa_t)$ comprises quality scores, a verdict, and confidence $\kappa_t \in \{0, 1, 2, 3, 4\}$, with

$$\mathbf{v}_t = (v_t^{\text{fmt}}, v_t^{\text{act}}, v_t^{\text{state}}, v_t^{\text{delta}}, v_t^{\text{progress}}, v_t^{\text{recovery}}, v_t^{\text{safe}}, v_t^{\text{eff}}) \in \{0, 1, 2, 3, 4\}^8. \quad (3)$$

These dimensions measure format validity, action validity, state grounding, delta evidence, task progress, recovery applicability, loop safety, and cost efficiency. Higher scores indicate better quality. Appendix A.1 specifies the contract constraints.

3.2 AUTOMATED DATA ANNOTATION AND GENERATION

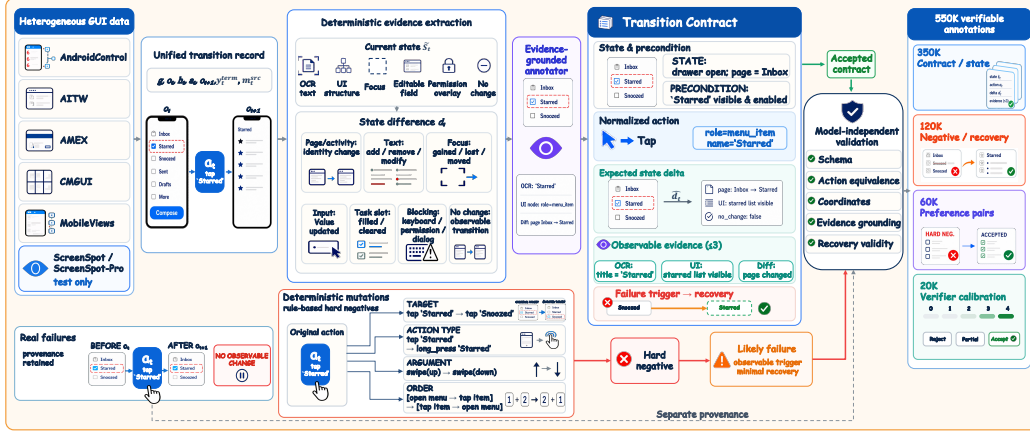


Figure 3: Automated construction of verifiable transition supervision. Deterministic state differencing, evidence-grounded annotation, counterfactual generation, and model-independent validation produce 550K annotations for contracts, states, negative actions, recovery, preferences, and verifier calibration.

Multimodal labeling and deterministic validation yield 550K annotations, as shown in Figure 3. We unify AndroidControl, AITW, AMEX, CMGUI, MobileViews, ScreenSpot, and ScreenSpot-Pro. ScreenSpot and ScreenSpot-Pro are reserved for Contract Policy and Verifier validation and testing, with no training use.

Contract annotation. Deterministic extraction yields state $\tilde{s}_t$ and difference d_t for each observed transition x_t . Multimodal annotation produces $c_t^+ = A_C(x_t, \tilde{s}_t, d_t)$. Acceptance requires demonstrated-action equivalence and valid structure, evidence, and recovery. State and preconditions use current observations. Subsequent observations supervise expected changes and evidence while remaining excluded from policy inputs. Static grounding samples supply localization evidence.

Negative, recovery, and calibration supervision. Rule-based action perturbations produce syntactically plausible hard negatives. The annotator predicts their likely failures, observable triggers, and minimal recovery branches. Observed and counterfactual failures retain separate provenance. Preference pairs contrast accepted contracts against negatives under the same decision input. Positive transitions, counterfactual negatives, and observed failures provide verifier calibration targets. Appendix A.2 details the transition representation and annotation procedure.

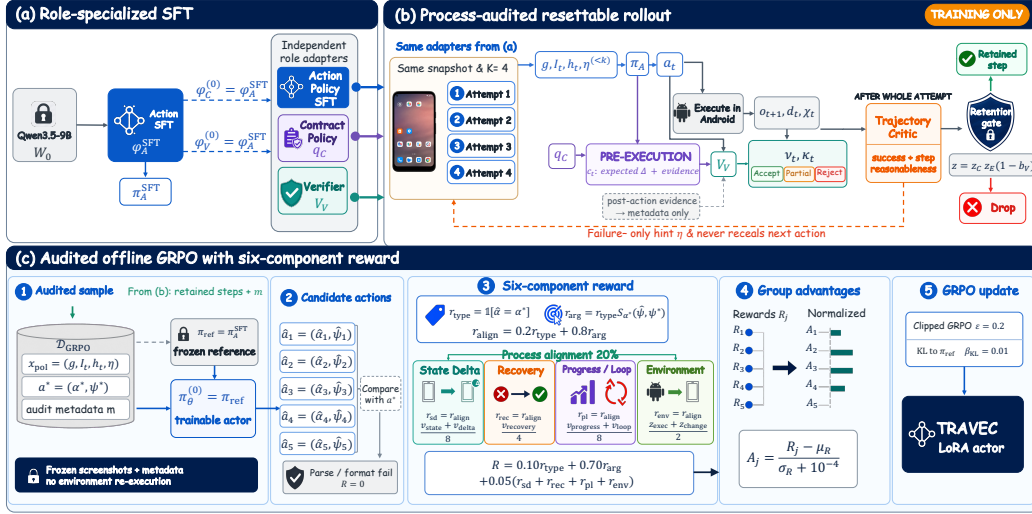


Figure 4: Three-stage policy optimization framework. Role-specialized SFT initializes the Contract and Verifier policies, resettable rollouts provide audited trajectories, and offline GRPO optimizes the Action Policy with six rewards.

3.3 SUPERVISED TRAINING OF ROLE-SPECIALIZED ADAPTERS

Each role uses a Qwen3.5-9B backbone W_0 with LoRA parameters ϕ , as shown in Figure 4. Action SFT learns ϕ_A^{SFT} by minimizing cross-entropy over normalized action completions with a frozen vision encoder. The auxiliary policies initialize from $\phi_C^{(0)} = \phi_V^{(0)} = \phi_A^{\text{SFT}}$ and train independently. The Contract Policy learns contracts and recovery from decision-time information. The Verifier learns post-action assessments from contracts, observed transitions, and validation indicators. Their parameters remain separate during rollout.

3.4 PROCESS-AUDITED OFFLINE GRPO

Rollout and audit. We collect $K = 4$ attempts per training task, resetting its initial AndroidWorld state each time, across 400 tasks. The Action Policy proposes an action, the Contract Policy predicts its contract, and the Verifier assesses the resulting transition. A fixed multimodal trajectory Critic uses the goal, actions, and visual execution evidence to assess feasibility, trajectory success $\hat{Y}_i^{(k)} \in \{0, 1\}$, and step reasonableness $z_{i,t}^{C,(k)} \in \{0, 1\}$.

Step selection combines Critic approval, execution validity, and a veto for high-confidence verifier rejection:

$$z_{i,t}^{(k)} = z_{i,t}^{C,(k)} z_{i,t}^{E,(k)} (1 - b_{i,t}^{V,(k)}), \quad b_{i,t}^{V,(k)} = \mathbf{1}[\nu_{i,t}^{(k)} = \text{reject} \wedge \kappa_{i,t}^{(k)} \geq 3]. \quad (4)$$

Here z^E indicates successful execution without errors or action invalidity. Critic hints $\eta_i^{(<k)}$ describe prior failures and behavior to avoid. They exclude correct next actions and are initially empty.

Offline sample selection. We retain tasks with Critic-assessed success rates in $[0, 0.9]$ across four attempts. For attempt length $T_i^{(k)}$, we define

$$q_i^{(k)} = \frac{\sum_t z_{i,t}^{(k)}}{\max(1, T_i^{(k)})}, \quad k_i^* = \arg \max_k^{\text{lex}} (\hat{Y}_i^{(k)}, q_i^{(k)}, -T_i^{(k)}). \quad (5)$$

Selection prioritizes success, a higher retained-step fraction, then shorter length. Retained steps from the selected attempt form

$$\mathcal{D}_{\text{GRPO}} = \{(x_{i,t}^{\text{pol}}, a_{i,t}^*, m_{i,t}) : z_{i,t}^{(k_i^*)} = 1\}. \quad (6)$$

The input x^{pol} contains the goal, screenshot, action history, and hints available at collection time. The reference a^* and audit evidence m remain fixed. Appendix A.3 specifies selection and hint handling.

Six-component reward. Following adaptive action scoring (Liu et al., 2026b), a candidate $\hat{a} = (\hat{\alpha}, \hat{\psi})$ is compared with the retained action $a^* = (\alpha^*, \psi^*)$:

$$r_{\text{type}} = \mathbf{1}[\hat{\alpha} = \alpha^*], \quad r_{\text{arg}} = r_{\text{type}} S_{\alpha^*}(\hat{\psi}, \psi^*), \quad r_{\text{align}} = 0.2r_{\text{type}} + 0.8r_{\text{arg}}. \quad (7)$$

S_{α^*} measures type-specific argument similarity. Invalid completions receive zero reward. Multiplication by r_{align} weights reference audit scores by candidate-reference similarity, reducing their contribution for dissimilar actions. The four process rewards are

$$r_{\text{sd}} = r_{\text{align}}(v^{\text{state}} + v^{\text{delta}})/8, \quad r_{\text{rec}} = r_{\text{align}}v^{\text{recovery}}/4, \quad (8)$$

$$r_{\text{pl}} = r_{\text{align}}(v^{\text{progress}} + v^{\text{safe}})/8, \quad r_{\text{env}} = r_{\text{align}}(e + u)/2. \quad (9)$$

Let $\mathcal{E}$ denote reported execution success and $\delta \in \{0, 1\}$ indicate an observable change in the selected transition. The environment indicators are

$$e = \mathbf{1}[\mathcal{E}], \quad u = \max\{\hat{Y}_i^{(k_i^*)}, \delta\}. \quad (10)$$

Selection through z^E additionally excludes execution errors and invalid actions. Change alone need not imply progress. Audit evidence is fixed per sample, so candidate dependence enters through r_{align} . This surrogate assumes local transfer of audit quality across similar actions.

The six rewards combine five verifier dimensions with direct action matching and recorded environment evidence:

$$R = w_{\text{type}}r_{\text{type}} + w_{\text{arg}}r_{\text{arg}} + w_{\text{sd}}r_{\text{sd}} + w_{\text{rec}}r_{\text{rec}} + w_{\text{pl}}r_{\text{pl}} + w_{\text{env}}r_{\text{env}}. \quad (11)$$

Appendices A.4 and B specify scores and weights.

Optimization and deployment. The actor and fixed reference are initialized from Action SFT. For each fixed input, the sampling policy $\pi_{\theta_{\text{old}}}$ generates $G = 5$ completions. Each completion receives reward $R_j = R(\hat{a}_j, a^*, m)$ and advantage $A_j = (R_j - \mu_R)/(\sigma_R + 10^{-4})$, where μ_R and σ_R are the mean and population standard deviation of the G rewards. With clipping radius $\epsilon = 0.2$ and KL coefficient $\beta_{\text{KL}} = 0.01$, we minimize

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E} \left[\frac{1}{G} \sum_{j=1}^G \frac{1}{L_j} \sum_{\ell=1}^{L_j} \left(\min\{\xi_{j,\ell} A_j, \text{clip}(\xi_{j,\ell}, 1 - \epsilon, 1 + \epsilon) A_j\} - \beta_{\text{KL}} D_{\text{KL}}^{j,\ell} \right) \right], \quad (12)$$

where L_j is completion length and $D_{\text{KL}}^{j,\ell}$ regularizes against the fixed SFT reference. The token probability ratio is

$$\xi_{j,\ell} = \frac{\pi_{\theta}(\hat{a}_{j,\ell} \mid x, \hat{a}_{j,<\ell})}{\pi_{\theta_{\text{old}}}(\hat{a}_{j,\ell} \mid x, \hat{a}_{j,<\ell})}. \quad (13)$$

GRPO samples actions on fixed observations and scores them using fixed audit evidence. Environment interaction occurs only during collection. Appendix A.5 gives further details. Deployment uses only $\pi_{\theta_{\text{A}}}^{\text{TRAVEC}}(a_t \mid g, I_t, h_t, \emptyset)$, which emits one action per step.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

We initialize TRAVEC from the post-trained Qwen3.5-9B multimodal model. Action SFT and auxiliary adapter training use rank-16 LoRA, while the subsequent GRPO stage uses rank-64 LoRA. The vision encoder is frozen throughout training. We evaluate TRAVEC-9B against seven publicly available GUI agents across four offline and two online benchmarks. Additional details on optimization hyperparameters are provided in Appendix B.

Benchmarks and metrics. We evaluate TRAVEC-9B on four offline and two online benchmarks. AndroidControl contains demonstrations from 833 Android applications with explicit instructions,

and we report action accuracy over 9,980 test steps per split (Li et al., 2024a). GUI-Odyssey evaluates cross-application mobile navigation over 83,885 steps spanning random, task, device, and app splits, with step success rate, type match, and grounding rate as the evaluation metrics (Lu et al., 2025a). ScreenSpot contains 1,272 grounding queries across mobile, desktop, and web interfaces (Cheng et al., 2024), while ScreenSpot-Pro contains 1,581 queries from high-resolution professional applications (Li et al., 2025a). We report target-hit accuracy on both benchmarks. AndroidWorld provides 116 reproducible task templates across 20 Android applications (Rawles et al., 2025). We evaluate three independently instantiated episodes per template and report episode success rate. MobileWorld evaluates long-horizon cross-application workflows with reproducible backend verification (Kong et al., 2026). Under a GUI-only interaction setting, we evaluate 117 tasks with one attempt per task and report pass-at-one success rate.

Comparison models. Qwen3.5-9B is a natively multimodal 9B foundation model with a hybrid linear and full-attention backbone and serves as our base model (Qwen Team, 2026). We compare TRAVEC-9B against seven publicly available GUI agents ranging from 7B to 8B parameters. Aguis-7B-720P is a cross-platform pure-vision agent trained with a two-stage grounding and planning pipeline (Xu et al., 2024). GUI-R1-7B applies GRPO with unified action-rule rewards on a curated dataset (Luo et al., 2025). ScaleCUA-7B is trained on large-scale cross-platform computer-use data synthesized across six operating systems (Liu et al., 2026d). GUI-Libra-8B combines action-aware supervision with KL-regularized, partially verifiable reinforcement learning (Yang et al., 2026). OpenMobile-8B uses mobile tasks and trajectories synthesized through targeted exploration and policy switching (Cheng et al., 2026). MemGUI-8B-SFT incorporates long-horizon interaction histories through a Context-as-Action framework (Liu et al., 2026a). ForgeQwen3-8B uses MobileForge to adapt the Qwen3-VL checkpoint with automatically generated target-app tasks and hierarchical step-wise feedback (Liu et al., 2026b).

4.2 DATA COMPOSITION AND SAMPLING

Figure 5 summarizes the composition of our 550,000 verifiable supervision items and their source distributions. The corpus consists of 63.64% transition-contract and state items, 21.82% negative-recovery items, 10.91% preference pairs, and 3.64% verifier-calibration items. Training supervision is constructed from AITW, the training split of AndroidControl, CMGUI, and AMEX, while MobileViews is used only for parser construction. ScreenSpot and ScreenSpot-Pro appear in Figure 5 only as evaluation-only datasets.

Train-test separation. We maintain benchmark-specific separation between training and evaluation data. AndroidControl supervision is constructed only from its training split, while all reported AndroidControl results use the held-out test split. ScreenSpot and ScreenSpot-Pro are reserved for validation and evaluation and do not contribute training examples. The 400 AndroidWorld tasks used for resettable rollout are disjoint from the task templates used for evaluation. MobileWorld is used only for evaluation. No evaluation examples are used for SFT, hard-case sampling, or GRPO.

To increase exposure to challenging transitions during training, we identify 43,751 hard-case items. Among them, 70.69% correspond to low-verifier-confidence cases, 21.77% to environment errors or warnings, and the remainder to popup, keyboard, input, or stale-state cases. These samples are upsampled during policy optimization to emphasize difficult transitions and execution failures.

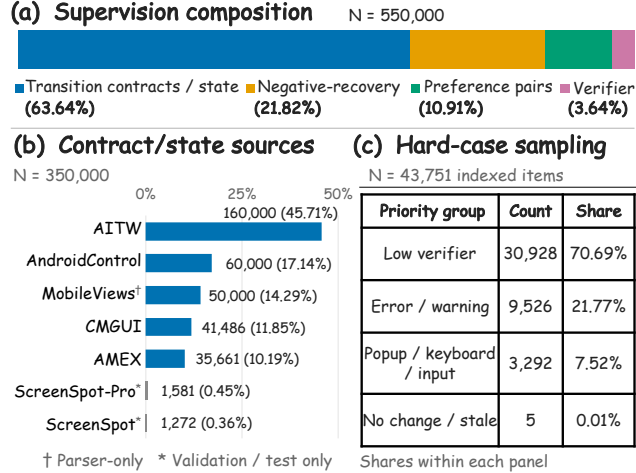


Figure 5: Supervision composition, data sources, and hard-case sampling. Parser-only and evaluation-only datasets are explicitly marked in panel (b).

4.3 OFFLINE BENCHMARK RESULTS

Table 1: Comparison with SOTA GUI agents across offline benchmarks and inference efficiency. High/Low denote action accuracy on AndroidControl under high-/low-level instructions. TM, GR, and SR denote type match, grounding rate, and step success rate on GUI-Odyssey. HT denotes target-hit accuracy on ScreenSpot and ScreenSpot-Pro (SS-Pro). Lat. and Tok. denote mean inference time and generated tokens per action. All models are evaluated under the same hardware and benchmark settings. Best results are shown in **bold**, and second-best results are underlined.

Model	AndroidControl		GUI-Odyssey			ScreenSpot	SS-Pro	Efficiency	
	High $\uparrow$	Low $\uparrow$	TM $\uparrow$	GR $\uparrow$	SR $\uparrow$	HT $\uparrow$	HT $\uparrow$	Lat. $\downarrow$	Tok. $\downarrow$
Aguvis-7B-720P	57.237	66.914	74.137	70.541	53.373	78.852	20.936	2.853	34.505
GUI-R1-7B	22.464	42.436	70.889	58.397	42.590	<u>85.849</u>	28.716	6.224	32.096
ScaleCUA-7B	36.056	46.608	77.218	69.886	57.315	<u>73.899</u>	23.340	3.591	35.800
GUI-Libra-8B	<u>62.856</u>	<u>81.417</u>	63.181	71.155	43.994	85.535	<u>53.700</u>	5.905	78.294
OpenMobile-8B	52.335	72.502	67.164	59.893	42.332	79.324	38.646	5.763	58.644
MemGUI-8B-SFT	58.061	77.424	68.932	62.140	45.444	76.965	39.216	12.910	318.260
ForgeQwen3-8B	62.665	79.850	79.864	71.221	<u>58.369</u>	82.704	46.490	7.070	86.799
Qwen3.5-9B	34.379	52.966	62.217	90.823	56.185	38.758	42.631	<u>0.815</u>	<u>18.039</u>
TRAVEC-9B (ours)	64.309	93.337	<u>79.588</u>	91.006	72.212	92.689	67.805	0.786	17.152

Table 1 shows consistent gains across action prediction, grounding, and step-level execution. TRAVEC-9B ranks first on six of the seven reported performance metrics, while remaining within 0.276 points of the best type-match result on GUI-Odyssey. The largest improvements appear on AndroidControl, where TRAVEC-9B reaches 64.309% on the High split and 93.337% on the Low split, outperforming Qwen3.5-9B by 29.930 and 40.371 percentage points, respectively. On the Low split, it also surpasses the best comparison model by 11.920 points. On GUI-Odyssey, TRAVEC-9B improves step success from 56.185% to 72.212% while preserving the strong grounding performance of the base model at 91.006%.

The gains extend to the grounding benchmarks and are accompanied by lower inference cost. TRAVEC-9B improves over Qwen3.5-9B by 53.931 points on ScreenSpot and 25.174 points on ScreenSpot-Pro, reaching 92.689% and 67.805%, respectively. On ScreenSpot-Pro, it exceeds the best comparison result by 14.105 points. Despite these accuracy gains, TRAVEC-9B remains more efficient than the compared models, requiring only 0.786 seconds and 17.152 generated tokens per action. Relative to the fastest specialized GUI baseline, it reduces latency by 72.4%, while also producing the shortest generations among all compared models.

4.4 ONLINE BENCHMARK RESULTS

Table 2 examines whether the improvements observed offline translate into reliable task completion in dynamic mobile environments. TRAVEC-9B achieves state-of-the-art performance on both AndroidWorld and MobileWorld. On AndroidWorld, it reaches a 35.345% episode success rate, exceeding its Qwen3.5-9B initialization by 14.655 percentage points and the strongest specialized baseline, ForgeQwen3-8B, by 0.862 points. On MobileWorld, TRAVEC-9B attains a 15.385% pass-at-one success rate, surpassing Qwen3.5-9B by 10.257 points and the second-best system, MemGUI-8B-SFT, by 4.274 points. Consistent improvements across two online bench-

Table 2: Comparison with SOTA GUI agents on online benchmarks. AndroidWorld uses episode success rate, whereas MobileWorld uses pass-at-one success rate. Best and second-best results are shown in **bold** and underlined.

Model	AndroidWorld	MobileWorld
	SR $\uparrow$	SR $\uparrow$
Aguvis-7B-720P	9.483	1.709
GUI-R1-7B	12.931	3.419
ScaleCUA-7B	14.655	3.419
GUI-Libra-8B	19.828	10.256
OpenMobile-8B	18.103	8.547
MemGUI-8B-SFT	25.000	<u>11.111</u>
ForgeQwen3-8B	<u>34.483</u>	5.983
Qwen3.5-9B	20.690	5.128
TRAVEC-9B (ours)	35.345	15.385

Table 4: Reward-weight ablation. Weights follow the order Type, Argument, State Delta, Recovery, Progress, and Environment. The final row reports the reward-weight configuration adopted by TRAVEC-9B. Best and second-best results are **bolded** and underlined.

Reward weights	AndroidControl		ScreenSpot	SS-Pro
	High ↑	Low ↑	HT ↑	HT ↑
0.125, 0.875, 0, 0, 0, 0	54.890	78.816	92.060	62.998
0.0625, 0.4375, 0.125, 0.125, 0.125, 0.125	<u>56.523</u>	82.425	92.453	<u>63.820</u>
0.025, 0.175, 0.2, 0.2, 0.2, 0.2	<u>55.631</u>	<u>82.745</u>	92.846	63.631
0, 0, 0.25, 0.25, 0.25, 0.25	55.952	81.152	92.060	63.188
0.10, 0.70, 0.05, 0.05, 0.05, 0.05	64.309	93.337	<u>92.689</u>	67.805

marks with distinct evaluation protocols show that transition-contract supervision and recovery-aware post-training translate beyond static action prediction to closed-loop execution. Together with the offline results, these findings indicate that explicitly modeling expected state transitions, verifying realized outcomes, and learning to recover from execution failures yields a policy that is more reliable in long-horizon interaction.

4.5 ABLATING PROCESS-AUDIT AND SUPERVISION COMPONENTS

Omitting Action SFT triggers severe degradation across every evaluation regime, demonstrating that audited GRPO fundamentally relies on robust structural priors to navigate the combinatorial GUI action space. Without this foundational alignment, subsequent process rewards lack the necessary baseline to shape coherent trajectories. Analyzing the auxiliary components reveals distinct functional complementarities. Disabling the Contract module or omitting its explicit state fields disproportionately impairs dense visual grounding on ScreenSpot-Pro. This indicates that forcing pre-execution state abstraction effectively binds visual interface elements to semantic text representations during training. Conversely, removing the Transition Verifier allows post-action hallucinations to permeate the training buffer, systematically eroding task-level reliability across dynamic environments. Omitting dynamic recovery supervision predictably exerts minimal impact on static metrics, as its empirical utility emerges exclusively within sequential error correction and loop avoidance visualized in Appendix C. Crucially, disabling trajectory Critic filtering yields worse performance despite retaining a significantly larger volume of rollout steps. This confirms that injecting noisy, serendipitous successes into the GRPO buffer actively corrupts policy alignment, proving that rigorous data curation heavily outweighs sheer exploration scale.

Table 3: Component ablations. Best and second-best results are **bolded** and underlined.

Model	AndroidControl		ScreenSpot	SS-Pro
	High ↑	Low ↑	HT ↑	HT ↑
w/o Critic	58.048	82.097	92.452	63.061
w/o Contract	57.024	80.110	<u>92.610</u>	63.820
w/o Verifier	<u>59.573</u>	82.994	92.374	63.947
w/o Contract Fields	56.344	82.816	91.824	62.998
w/o Recovery	56.723	<u>86.553</u>	92.296	<u>64.643</u>
w/o Action SFT	49.453	74.170	89.937	56.610
TRAVEC-9B	64.309	93.337	92.689	67.805

4.6 ABLATING PROCESS-REWARD WEIGHT

Table 4 exposes a strictly nonmonotonic trade-off between explicit action fidelity and environment-driven process shaping. The pure-action baseline enforces rigid demonstration mimicry but fails to internalize the actual state consequences, yielding sub-optimal generalization across dynamic trajectories. Conversely, shifting the entirety of the reward mass to the process dimensions severely destabilizes the optimization landscape. Without a strong action-matching anchor, the policy rapidly drifts toward invalid syntax and inefficient exploration, attempting to exploit state changes without executing coherent functional steps. Mixed reward objectives consistently outperform these extremes by structurally coupling syntactic correctness with verifiable environmental feedback. Modulating the process reward mass predictably dictates the behavioral prior of the agent, shifting the optimization focus between conservative instruction following and aggressive state manipulation.

The deployed TRAVEC-9B configuration strictly bounds process signals to twenty percent of the total reward mass while preserving action correctness as the primary gradient anchor. This specific architectural balance functions as a robust regularizer, safely grounding syntactic actions in real-world consequences while effectively suppressing reward hacking.

5 CONCLUSION

We presented TRAVEC, a framework that transforms transition prediction, evidence-based verification, and failure recovery into scalable annotation and reinforcement learning signals for mobile GUI agents. The automated pipeline synthesizes 550K verifiable contract, negative-recovery, preference, and calibration annotations. Leveraging this structured corpus, two auxiliary adapters and a trajectory Critic systematically audit resettable self-generated experiences, while a six-dimensional reward formulation aligns the final policy with action correctness, observed state changes, failure recovery, progress, loop avoidance, and execution outcomes. At deployment, the resulting TRAVEC-9B completely bypasses all auxiliary modules to strictly emit executable actions. Comprehensive evaluations demonstrate substantial improvements over the Qwen3.5-9B foundation and Action SFT baselines on AndroidWorld, alongside highly competitive performance across offline action and grounding benchmarks. Furthermore, systematic ablations reveal critical task-dependent trade-offs within the reward design, and matched efficiency tests confirm that our architecture achieves significantly lower latency and shorter generation lengths than the base model. More broadly, TRAVEC establishes a robust paradigm for migrating expensive reasoning and verification away from the online critical path and directly into auditable policy learning.

AI USE STATEMENT

Generative AI tools were used only to improve the writing, grammar, and presentation of this manuscript. All research ideas, methods, experiments, analyses, and conclusions were developed and verified by the authors.

ETHICS STATEMENT

This work uses existing research datasets and controlled benchmark environments. We do not collect additional personal user data or conduct new studies involving human participants.

REPRODUCIBILITY STATEMENT

We provide detailed descriptions of the model architecture, training procedure, data construction pipeline, reward formulation, evaluation protocols, and hyperparameter configurations in the main paper and appendix. These details are intended to enable faithful reproduction of our training and evaluation setup. Upon acceptance, we will release the code and constructed dataset associated with this work.

REFERENCES

- Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Wang. Agent s: An open agentic framework that uses computers like a human. In *International Conference on Learning Representations*, 2025a. URL <https://arxiv.org/pdf/2410.08164>.
- Saaket Agashe, Kyle Wong, Vincent Tu, Jiachen Yang, Ang Li, and Xin Eric Wang. Agent s2: A compositional generalist-specialist framework for computer use agents. *arXiv preprint arXiv:2504.00906*, 2025b. URL <https://arxiv.org/pdf/2504.00906>.
- Cong Chen, Kaixiang Ji, Hao Zhong, Muzhi Zhu, Anzhou Li, Guo Gan, Ziyuan Huang, Cheng Zou, Jiajia Liu, Jingdong Chen, et al. Gui-shepherd: Reliable process reward and verification for long-sequence gui tasks. *arXiv preprint arXiv:2509.23738*, 2025.
- Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. Seeclick: Harnessing gui grounding for advanced visual gui agents. In *Annual Meeting of the Association for Computational Linguistics*, 2024. URL <https://aclanthology.org/2024.acl-long.505.pdf>.
- Kanzhi Cheng, Zehao Li, Zheng Ma, Nuo Chen, Jialin Cao, Qiushi Sun, Zichen Ding, Fangzhi Xu, Hang Yan, Jiajun Chen, et al. Openmobile: Building open mobile agents with task and trajectory synthesis. *arXiv preprint arXiv:2604.15093*, 2026. URL <https://arxiv.org/pdf/2604.15093>.
- Zihan Dong, Rui Qian, Qishi Zhan, Dongshen Peng, Kaixin Li, and Yu Li. Why are gui agents correct but late? decode on the decision-time critical path, tested with pre-compiled policy trees. *arXiv preprint arXiv:2607.28399*, 2026. URL <https://arxiv.org/pdf/2607.28399>.
- Boyu Gou, Demi Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for gui agents. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/pdf/2410.05243>.
- Linqiang Guo, Wei Liu, Li Gu, Yang Wang, et al. Stepreflect: Structured ui transition reflection for mobile gui agents. *arXiv preprint arXiv:2608.05587*, 2026a.
- Linqiang Guo, Wei Liu, Yi Wen Heng, Tse-Hsun Peter Chen, and Yang Wang. Agent-sama: state-aware mobile assistant. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 29459–29467, 2026b.

- Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. Cogagent: A visual language model for gui agents. In *Conference on Computer Vision and Pattern Recognition*, 2024. URL <https://arxiv.org/pdf/2312.08914>.
- Kung-Hsiang Huang, Haoyi Qiu, Yutong Dai, Caiming Xiong, and Chien-Sheng Wu. Gui-kv: Efficient gui agents via kv cache with spatio-temporal awareness. *arXiv preprint arXiv:2510.00536*, 2025. URL <https://arxiv.org/pdf/2510.00536>.
- Runxi Huang, Liyu Zhang, Shengzhong Liu, and Xiaomin Ouyang. Mobileexplorer: Accelerating on-device inference for mobile gui agents via online exploration. *arXiv preprint arXiv:2605.26546*, 2026. URL <https://arxiv.org/pdf/2605.26546>.
- Quyu Kong, Xu Zhang, Zhenyu Yang, Nolan Gao, Chen Liu, Panrong Tong, Chenglin Cai, Hanzhang Zhou, Jianan Zhang, Liangyu Chen, et al. Mobileworld: Benchmarking autonomous mobile agents in agent-user interactive and mcp-augmented environments. In *Annual Meeting of the Association for Computational Linguistics*, 2026. URL <https://aclanthology.org/2026.acl-long.278.pdf>.
- Hanyu Lai, Xiao Liu, Yanxiao Zhao, Han Xu, Hanchen Zhang, Bohao Jing, Yanyu Ren, Shuntian Yao, Yuxiao Dong, and Jie Tang. Computerrl: Scaling end-to-end online reinforcement learning for computer use agents. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/pdf/2508.14040>.
- Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. Screenspot-pro: Gui grounding for professional high-resolution computer use. In *International Conference on Multimedia*, 2025a. URL <https://arxiv.org/pdf/2504.07981>.
- Shufan Li, Konstantinos Kallidromitis, Akash Gokul, Yusuke Kato, Kazuki Kozuka, and Aditya Grover. Mobileworldbench: Towards semantic world modeling for mobile agents. *arXiv preprint arXiv:2512.14014*, 2025b.
- Wei Li, William Bishop, Alice Li, Chris Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on ui control agents. In *Advances in Neural Information Processing Systems*, 2024a. URL <https://arxiv.org/pdf/2406.03679>.
- Yanda Li, Chi Zhang, Wenjia Jiang, Wanqi Yang, Bin Fu, Pei Cheng, Xin Chen, Ling Chen, and Yunchao Wei. Appagent v2: Advanced agent for flexible mobile interactions. *arXiv preprint arXiv:2408.11824*, 2024b. URL <https://arxiv.org/pdf/2408.11824>.
- Zehao Li, Zhenyu Wu, Yibo Zhao, Bowen Yang, Jingjing Xie, Zhaoyang Liu, Zhoumianze Liu, Kaiming Jin, Jianze Liang, Zonglin Li, Feng Wu, Bowen Zhou, Zun Wang, and Zichen Ding. Os-themis: A scalable critic framework for generalist gui rewards, 2026. URL <https://arxiv.org/abs/2603.19191>.
- Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Weixian Lei, Lijuan Wang, and Mike Zheng Shou. Showui: One vision-language-action model for gui visual agent, 2024. URL <https://arxiv.org/abs/2411.17465>.
- Guangyi Liu, Gao Wu, Congxiao Liu, Pengxiang Zhao, Liang Liu, Mading Li, Qi Zhang, Mengyan Wang, Liang Guo, and Yong Liu. Memgui-agent: An end-to-end long-horizon mobile gui agent with proactive context management. *arXiv preprint arXiv:2606.19926*, 2026a. URL <https://arxiv.org/pdf/2606.19926>.
- Guangyi Liu, Pengxiang Zhao, Gao Wu, Yiwen Yin, Mading Li, Liang Liu, Congxiao Liu, Zhang Qi, Mengyan Wang, Liang Guo, et al. Mobileforge: Annotation-free adaptation for mobile gui agents with hierarchical feedback-guided policy optimization. *arXiv preprint arXiv:2606.19930*, 2026b. URL <https://arxiv.org/pdf/2606.19930>.
- Xiangyan Liu, Kaixin Li, Haonan Wang, Biao Wu, Meng Fang, Longxu Dou, Chao Du, Michael Qizhe Shieh, and Tianyu Pang. Scaling gui agents with visual state transitions. *arXiv preprint arXiv:2607.24112*, 2026c.

- Zhaoyang Liu, JingJing Xie, Zichen Ding, Zehao Li, Bowen Yang, Zhenyu Wu, Xuehui Wang, Qiushi Sun, Shi Liu, Weiyun Wang, et al. Scalecua: Scaling open-source computer use agents with cross-platform data. In *International Conference on Learning Representations*, 2026d. URL <https://arxiv.org/pdf/2509.15221>.
- Quanfang Lu, Wenqi Shao, Zitao Liu, Lingxiao Du, Fanqing Meng, Boxuan Li, Botong Chen, Siyuan Huang, Kaipeng Zhang, and Ping Luo. Guidyssey: A comprehensive dataset for cross-app gui navigation on mobile devices. In *International Conference on Computer Vision*, 2025a. URL <https://arxiv.org/pdf/2406.08451>.
- Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. Omniparser for pure vision based gui agent, 2024. URL <https://arxiv.org/abs/2408.00203>.
- Zhengxi Lu, Jiabo Ye, Fei Tang, Yongliang Shen, Haiyang Xu, Ziwei Zheng, Weiming Lu, Ming Yan, Fei Huang, Jun Xiao, and Yueting Zhuang. Ui-s1: Advancing gui automation via semi-online reinforcement learning, 2025b. URL <https://arxiv.org/abs/2509.11543>.
- Zhengxi Lu, Fei Tang, Guangyi Liu, Jin Ma, Kaitao Song, Xu Tan, Wenqi Zhang, Weiming Lu, Jun Xiao, Yueting Zhuang, et al. Ui-copilot: Advancing long-horizon gui automation via tool-integrated policy optimization. In *Annual Meeting of the Association for Computational Linguistics*, 2026. URL <https://aclanthology.org/2026.acl-long.904.pdf>.
- Run Luo, Lu Wang, Wanwei He, Longze Chen, Jiaming Li, and Xiaobo Xia. Gui-r1: A generalist r1-style vision-language action model for gui agents. *arXiv preprint arXiv:2504.10458*, 2025. URL <https://arxiv.org/pdf/2504.10458>.
- Jian Mu, Chaoyun Zhang, Chiming Ni, Lu Wang, Bo Qiao, Kartik Mathur, Qianhui Wu, Yuhang Xie, Xiaojun Ma, Mengyu Zhou, et al. GUI-360°: A comprehensive dataset and benchmark for computer-using agents. *arXiv preprint arXiv:2511.04307*, 2025. URL <https://arxiv.org/pdf/2511.04307>.
- Jiachen Qian. Viscritic: Visual state comparison as process reward for gui agents. *arXiv preprint arXiv:2606.24525*, 2026.
- Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025. URL <https://arxiv.org/pdf/2501.12326>.
- Qwen Team. Qwen3.5-9b. Hugging Face model repository, 2026. URL <https://huggingface.co/Qwen/Qwen3.5-9B>. Accessed 2026-08-26.
- Chris Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, et al. Androidworld: A dynamic benchmarking environment for autonomous agents. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/pdf/2405.14573>.
- Qiushi Sun, Kanzhi Cheng, Zichen Ding, Chuanyang Jin, Yian Wang, Fangzhi Xu, Zhenyu Wu, Chengyou Jia, Liheng Chen, Zhoumianze Liu, et al. Os-genesis: Automating gui agent trajectory construction via reverse task synthesis. In *Annual Meeting of the Association for Computational Linguistics*, 2025. URL <https://aclanthology.org/2025.acl-long.277.pdf>.
- Fei Tang, Haolei Xu, Hang Zhang, Siqi Chen, Xingyu Wu, Yongliang Shen, Wenqi Zhang, Guiyang Hou, Zeqi Tan, Yuchen Yan, et al. A survey on (m) llm-based gui agents. *arXiv preprint arXiv:2504.13865*, 2025. URL <https://arxiv.org/abs/2504.13865>.
- Haoming Wang, Haoyang Zou, Huatong Song, Jiazhan Feng, Junjie Fang, Juntong Lu, Longxiang Liu, Qinyu Luo, Shihao Liang, Shijue Huang, et al. Ui-tars-2 technical report: Advancing gui agent with multi-turn reinforcement learning. *arXiv preprint arXiv:2509.02544*, 2025. URL <https://arxiv.org/pdf/2509.02544>.

- Junyang Wang, Haiyang Xu, Haitao Jia, Xi Zhang, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. Mobile-agent-v2: Mobile device operation assistant with effective navigation via multi-agent collaboration. In *Advances in Neural Information Processing Systems*, 2024a. URL <https://arxiv.org/pdf/2406.01014>.
- Shuai Wang, Weiwen Liu, Jingxuan Chen, Yuqi Zhou, Weinan Gan, Xingshan Zeng, Yuhan Che, Shuai Yu, Xinlong Hao, Kun Shao, et al. Gui agents with foundation models: A comprehensive survey. *arXiv preprint arXiv:2411.04890*, 2024b. URL <https://arxiv.org/abs/2411.04890>.
- Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, et al. Os-atlas: Foundation action model for generalist gui agents. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/pdf/2410.23218>.
- Han Xiao, Guozhi Wang, Hao Wang, Shilong Liu, Yuxiang Chai, Yue Pan, Yufeng Zhou, Xiaoxin Chen, Yafei Wen, and Hongsheng Li. Ui-mem: Self-evolving experience memory for online reinforcement learning in mobile gui agents, 2026. URL <https://arxiv.org/abs/2602.05832>.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, 2024. URL <https://arxiv.org/pdf/2404.07972>.
- Haiyang Xu, Xi Zhang, Haowei Liu, Junyang Wang, Zhaozai Zhu, Shengjie Zhou, Xuhao Hu, Feiyu Gao, Junjie Cao, Zihua Wang, Zhiyuan Chen, Jitong Liao, Qi Zheng, Jiahui Zeng, Ze Xu, Shuai Bai, Junyang Lin, Jingren Zhou, and Ming Yan. Mobile-agent-v3.5: Multi-platform fundamental gui agents, 2026. URL <https://arxiv.org/abs/2602.16855>.
- Yifan Xu, Xiao Liu, Xinghan Liu, Jiaqi Fu, Hanchen Zhang, Bohao Jing, Shudan Zhang, Yuting Wang, Wenyi Zhao, and Yuxiao Dong. Mobilerl: Online agentic reinforcement learning for mobile gui agents, 2025. URL <https://arxiv.org/abs/2509.18119>.
- Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguvis: Unified pure vision agents for autonomous gui interaction. *arXiv preprint arXiv:2412.04454*, 2024. URL <https://arxiv.org/pdf/2412.04454>.
- Rui Yang, Qianhui Wu, Zhaoyang Wang, Hanyang Chen, Ke Yang, Hao Cheng, Huaxiu Yao, Baolin Peng, Huan Zhang, Jianfeng Gao, et al. Gui-libra: Training native gui agents to reason and act with action-aware supervision and partially verifiable rl. *arXiv preprint arXiv:2602.22190*, 2026. URL <https://arxiv.org/pdf/2602.22190>.
- Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, Jitong Liao, Qi Zheng, Fei Huang, Jingren Zhou, and Ming Yan. Mobile-agent-v3: Fundamental agents for gui automation, 2025. URL <https://arxiv.org/abs/2508.15144>.
- Keen You, Haotian Zhang, Eldon Schoop, Floris Weers, Amanda Swearngin, Jeffrey Nichols, Yinfei Yang, and Zhe Gan. Ferret-ui: Grounded mobile ui understanding with multimodal llms. In *European Conference on Computer Vision*, 2024. URL <https://arxiv.org/pdf/2404.05719>.
- Shengcheng Yu, Yuchen Ling, Junyang Xing, Quan Zhou, Chunrong Fang, and Zhenyu Chen. Software engineering for and with gui agent. *arXiv preprint arXiv:2608.09278*, 2026. URL <https://arxiv.org/pdf/2608.09278>.
- Yuzhe Zhang, Xianwei Xue, Xingyong Wu, Mengke Chen, Chen Liu, Xinran He, Run Shao, Feiran Liu, Huanmin Xu, Qitong Pan, et al. Don’t act blindly: Robust gui automation via action-effect verification and self-correction. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 28924–28941, 2026.

Zhong Zhang, Yaxi Lu, Yikun Fu, Yupeng Huo, Shenzhi Yang, Yesai Wu, Han Si, Xin Cong, Hao-tian Chen, Yankai Lin, et al. Agentcpm-gui: Building mobile-use agents with reinforcement fine-tuning. In *Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2025. URL <https://aclanthology.org/2025.emnlp-demos.12.pdf>.

Hanzhang Zhou, Panrong Tong, Xu Zhang, Quyu Kong, Chenglin Cai, Tianyu Xia, Gongjie Zhang, Jianan Zhang, Long Li, Long Chen, et al. Qwen-ui-agent technical report: Toward next-generation real-world centric foundation gui agents. *arXiv preprint arXiv:2607.28227*, 2026a. URL <https://arxiv.org/pdf/2607.28227>.

Xurui Zhou, Gongwei Chen, Yuquan Xie, Zaijing Li, Kaiwen Zhou, Shuai Wang, Shuo Yang, Zhuotao Tian, and Rui Shao. Hiconagent: History context-aware policy optimization for gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026b. URL https://openaccess.thecvf.com/content/CVPR2026/papers/Zhou_HiconAgent_History_Context-aware_Policy_Optimization_for_GUI_Agents_CVPR_2026_paper.pdf.

CONTENTS OF APPENDIX

A	Supplementary Method Details	16
A.1	Transition contracts and verifier scores	16
A.2	Annotation representation and procedure	17
A.3	Trajectory auditing, hints, and selection	17
A.4	Action similarity and environment evidence	17
A.5	Group-relative optimization	18
B	Experimental Configuration and Implementation Details	18
B.1	Implementation and training configuration	18
B.2	Reward weights	18
B.3	Benchmarks and metrics	18
B.4	Compared models	19
C	Case Studies on Transition, Drift, and Recovery	19
C.1	Silent transition failure	19
C.2	State drift and termination	19
C.3	Failure recovery	20
D	Annotated Samples	20

A SUPPLEMENTARY METHOD DETAILS

A.1 TRANSITION CONTRACTS AND VERIFIER SCORES

The action history is $h_t = (a_1, \dots, a_{t-1})$. Action types include click, long press, swipe, type, key or system button, open, wait, and terminate. Arguments are type-specific and spatial coordinates are normalized. A trajectory is $\tau = (g, o_1, a_1, \dots, o_T, a_T, o_{T+1})$.

The deterministic difference $d_t = \mathcal{D}(o_t, o_{t+1})$ tracks page or activity identity, text, focus, input values, task variables, and blocking conditions, including the absence of observable change. A contract contains up to three checkable evidence queries E_t . Its optional recovery specification is

$$\rho_t = (\phi_t, a_t^{\text{rec}}, \hat{d}_t^{\text{rec}}, E_t^{\text{rec}}), \quad (14)$$

where ϕ_t is the failure trigger, a_t^{rec} the corrective action, $\hat{d}_t^{\text{rec}}$ the expected change, and E_t^{rec} the supporting evidence. Validation indicators χ_t assess structural validity, coordinate bounds, admissible action types, and consistency with the proposed action.

The Verifier returns quality scores, a verdict $\nu_t \in \{\text{accept}, \text{partial}, \text{reject}\}$, and confidence $\kappa_t \in \{0, 1, 2, 3, 4\}$. Its score vector is

$$\mathbf{v}_t = (v_t^{\text{fmt}}, v_t^{\text{act}}, v_t^{\text{state}}, v_t^{\text{delta}}, v_t^{\text{progress}}, v_t^{\text{recovery}}, v_t^{\text{safe}}, v_t^{\text{eff}}) \in \{0, 1, 2, 3, 4\}^8. \quad (15)$$

The dimensions evaluate format validity, action validity, state grounding, delta evidence, task progress, recovery applicability, loop safety, and cost efficiency. Higher scores indicate better quality. Loop safety measures avoidance of repeated ineffective actions, decisions based on outdated observations, and cyclic retries. Five dimensions enter the process rewards. Direct action matching supplies r_{type} and r_{arg} , while recorded environment evidence supplies r_{env} .

A.2 ANNOTATION REPRESENTATION AND PROCEDURE

The unified transition representation is

$$x_t = (g, o_t, h_t, a_t, o_{t+1}, y_t^{\text{term}}, m_t^{\text{src}}), \quad (16)$$

where y_t^{term} is the available terminal label and m_t^{src} identifies provenance. Next observations and transition differences are available only for samples with observed transitions. Static grounding samples provide localization evidence.

Claude Opus 4.8 annotates transitions using the deterministic state estimate $\bar{s}_t$ and observed difference d_t . Contract acceptance requires semantic action equivalence and valid structure, evidence, and recovery specifications. For each accepted contract, rule-based perturbations of the target or argument generate up to four syntactically plausible hard negatives. The annotator specifies the likely failure, an observable trigger, and a minimal recovery branch. Observed and counterfactual failures retain separate provenance. Preference pairs share the same decision input, and calibration targets cover all eight verifier dimensions.

A.3 TRAJECTORY AUDITING, HINTS, AND SELECTION

The trajectory Critic uses a fixed Qwen3.5-9B model without a task-specific adapter. It first describes each action and its adjacent screenshot evidence, then assesses feasibility, trajectory success, and every step’s reasonableness using these descriptions, execution records, and up to the last three screenshots. After an attempt containing failures or rejected steps, it generates hints restricted to observed failures and behavior to avoid. Correct next actions are excluded from the hints.

An attempt generates hints when the task fails, any step is judged unreasonable, or any step is rejected by the execution or verifier checks. Hints accumulate across attempts as $\eta_i^{(<k)}$. The first attempt has empty hints. Each offline sample preserves the hints available when its reference action was collected. Deployment sets this input to $\emptyset$.

For the retention criterion, z^E requires a successful execution response with no execution error or action invalidity. A verifier rejection vetoes a step only when confidence is at least 3. An invalid verifier assessment gives $b^V = 0$.

We compute task success from all $K = 4$ attempts and retain tasks satisfying

$$\frac{1}{K} \sum_{k=1}^K \hat{Y}_i^{(k)} \in [0, 0.9]. \quad (17)$$

For attempt length $T_i^{(k)}$, the retained-step fraction and selected attempt are

$$q_i^{(k)} = \frac{\sum_t z_{i,t}^{(k)}}{\max(1, T_i^{(k)})}, \quad k_i^* = \arg \max_k (\hat{Y}_i^{(k)}, q_i^{(k)}, -T_i^{(k)}). \quad (18)$$

Complete ties favor the earlier attempt. The policy input for each retained action is

$$x_{i,t}^{\text{pol}} = (g_i, I_{i,t}, h_{i,t}, \eta_i^{(<k_i^*)}). \quad (19)$$

Audit evidence $m_{i,t}$ contains the contract, verifier scores, post-action observation, execution outcome, and Critic judgment. These AndroidWorld rollouts supply reinforcement-learning experience separately from the annotation sources.

A.4 ACTION SIMILARITY AND ENVIRONMENT EVIDENCE

The type-specific similarity S_{α^*} uses distance-decayed point similarity for clicks and long presses, direction matching for swipes, token-set F1 for typing, exact or containment matching for application names, and type-specific argument matching otherwise.

Let $\mathcal{E}_{i,t}^{(k)}$ denote the event that the environment reports successful action execution, and let $\delta_{i,t}^{(k)} \in \{0, 1\}$ indicate an observable change identified by $\mathcal{D}$. For the selected attempt,

$$e = \mathbf{1}[\mathcal{E}_{i,t}^{(k_i^*)}], \quad u = \max\{\hat{Y}_i^{(k_i^*)}, \delta_{i,t}^{(k_i^*)}\}. \quad (20)$$

The reward indicator e records the success response, while step selection through z^E additionally excludes execution errors and action invalidity. The indicator u combines Critic-assessed trajectory success with observable change. Change alone need not imply task progress.

All process scores belong to the executed reference transition. Multiplication by r_{align} scales their contribution by candidate-reference similarity under a local transfer assumption. The fixed audit scores introduce no candidate dependence beyond this action alignment.

A.5 GROUP-RELATIVE OPTIMIZATION

The actor and fixed reference are initialized from the Action SFT policy:

$$\pi_{\theta(0)} = \pi_{\text{ref}} = \pi_{\theta_A^{\text{SFT}}}. \quad (21)$$

For each fixed input x , the sampling policy generates $G = 5$ completions with rewards $R_j = R(\hat{a}_j, a^*, m)$. Their group statistics and advantages are

$$\mu_R = \frac{1}{G} \sum_j R_j, \quad \sigma_R^2 = \frac{1}{G} \sum_j (R_j - \mu_R)^2, \quad A_j = \frac{R_j - \mu_R}{\sigma_R + 10^{-4}}. \quad (22)$$

For completion length L_j and token index ℓ , the probability ratio is

$$\xi_{j,\ell} = \frac{\pi_{\theta}(\hat{a}_{j,\ell} \mid x, \hat{a}_{j,<\ell})}{\pi_{\theta_{\text{old}}}(\hat{a}_{j,\ell} \mid x, \hat{a}_{j,<\ell})}. \quad (23)$$

The sampling policy supplies the ratio denominator, and the fixed SFT reference supplies KL regularization at the same token context. We use clipping radius $\epsilon = 0.2$ and KL coefficient $\beta_{\text{KL}} = 0.01$. Policy optimization resamples actions on fixed observations and computes rewards from the reference action and audit evidence. Environment execution and auxiliary auditing occur during data collection.

B EXPERIMENTAL CONFIGURATION AND IMPLEMENTATION DETAILS

B.1 IMPLEMENTATION AND TRAINING CONFIGURATION

We utilize the post-trained Qwen3.5-9B multimodal model (Qwen Team, 2026) as the common base. Action, Contract, and Verifier SFT deploy rank-16 LoRA for two epochs guided by respective learning rates of 2×10^{-5} , 10^{-6} , and 10^{-6} . GRPO leverages rank 64, four epochs, a learning rate of 10^{-6} , $G = 5$, a KL coefficient of 0.01, and a clipping radius of 0.2. Formal distributed training runs across seven 8-GPU nodes equipped with EFA; evaluation scales across 15 8-GPU nodes utilizing a single tensor-parallel-size-1 vLLM instance per GPU. External checkpoints strictly follow their released prompts, parsers, coordinate conventions, and native direct-action pathways. We enforce full sample reporting, rigidly reject runs exhibiting missing samples or API errors, and retain raw predictions alongside per-task records.

B.2 REWARD WEIGHTS

The reward weights follow the order Action Type, Action Argument, State Delta, Recovery, Progress/Loop, and Online Environment:

$$(w_{\text{type}}, w_{\text{arg}}, w_{\text{sd}}, w_{\text{rec}}, w_{\text{pl}}, w_{\text{env}}) = (0.1, 0.7, 0.05, 0.05, 0.05, 0.05). \quad (24)$$

Direct action matching contributes 80% of the total weight, and the four process rewards contribute 20%. The coefficients 0.2 and 0.8 in r_{align} define candidate-reference similarity within each process reward.

Trajectory auditing is detailed in Appendix A.3.

B.3 BENCHMARKS AND METRICS

We rigorously evaluate four offline and two executable benchmarks. AndroidControl aggregates diverse demonstrations from 833 Android applications providing explicit instructions; we report

action accuracy across 9,980 test steps per split (Li et al., 2024a). GUI-Odyssey specifically targets cross-application mobile navigation; its evaluation suite encompasses 83,885 steps spanning random, task, device, and app splits, where we report overall step success rate, type match, and grounding rate (Lu et al., 2025a). ScreenSpot consolidates 1,272 grounding queries across mobile, desktop, and web interfaces (Cheng et al., 2024), whereas ScreenSpot-Pro curates 1,581 queries sourced from high-resolution professional applications (Li et al., 2025a); both rigorously measure target-hit accuracy. AndroidWorld supplies 116 reproducible task templates spanning 20 Android applications (Rawles et al., 2025); we execute three separate instances per template reporting the episode success rate. MobileWorld stresses heavily extended, cross-application workflows bounded by reproducible backend verification (Kong et al., 2026); abiding by our strict GUI-only protocol, we evaluate 117 tasks running one attempt each to report the pass-at-one success rate.

B.4 COMPARED MODELS

Qwen3.5-9B operates as a natively multimodal 9B foundation model architected with a hybrid linear and full-attention language backbone, serving as our unadapted base control (Qwen Team, 2026). We benchmark against seven widely released 7B to 8B GUI agents. Aguis-7B-720P represents a cross-platform pure-vision agent optimized via a two-stage grounding and planning pipeline (Xu et al., 2024). GUI-R1-7B deploys GRPO paired with unified action-rule rewards applied to a heavily curated, compact dataset (Luo et al., 2025). ScaleCUA-7B directly scales cross-platform computer-use data synthesized over six distinct operating systems (Liu et al., 2026d). GUI-Libra-8B fuses action-aware supervision with KL-regularized, partially verifiable RL dynamics (Yang et al., 2026). OpenMobile-8B artificially synthesizes mobile tasks and trajectories leveraging targeted exploration and rapid policy switching (Cheng et al., 2026). MemGUI-8B-SFT proactively folds long-horizon interaction histories utilizing a Context-as-Action framework (Liu et al., 2026a). ForgeQwen3-8B adapts the Qwen3-VL checkpoint through MobileForge driven by automatically generated target-app tasks and hierarchical step-wise feedback (Liu et al., 2026b).

C CASE STUDIES ON TRANSITION, DRIFT, AND RECOVERY

Figure 6 contrasts unexecuted model predictions against ground-truth expert trajectories to expose three structural failure modes inherent to purely action-driven GUI agents.

C.1 SILENT TRANSITION FAILURE

Panel a visualizes a stall scenario where the agent must save a specific location to a shared category. Rather than executing the backward navigation and leftward swipe necessary to expose the hidden Save interface, the model predicts a syntactically valid but semantically useless tap targeting the map display controls. The expert trajectory reveals that navigating complex UI hierarchies demands deliberate physical exploration to uncover occluded states. This discrepancy empirically confirms that optimizing exclusively for localized visual grounding heavily biases the agent toward interacting with immediately salient elements, leaving the policy fundamentally incapable of reasoning through hidden multi-step state transitions.

C.2 STATE DRIFT AND TERMINATION

Panel b illustrates state drift caused by over-execution, contrasting a redundant model prediction against an expert termination immediately following timetable rendering. By Step 5, the agent effectively navigates the route hierarchy to fulfill the user instruction. Despite achieving this terminal goal state, the model triggers an extraneous tap targeting the operator icon. This structural overstepping demonstrates a critical flaw in standard behavioral cloning paradigms, which inherently condition agents toward continuous interaction as long as actionable elements remain on screen. This specific failure explicitly validates the necessity of integrating structured progress and environment rewards to instill precise, observation-driven stopping criteria.

C.3 FAILURE RECOVERY

Panel c demonstrates semantic recovery from an environmental trap, contrasting a generic system-level backward action with a precise in-app navigation and dialog-clearing sequence. Trapped inside an unrelated mathematical exercise, the model naively predicts a system back command. This represents a generic heuristic that frequently resets overarching task progress unpredictably across different mobile applications. Conversely, the expert executes a deliberate semantic recovery by targeting the native in-app back button, which intentionally surfaces a confirmation dialog. The expert subsequently clears this blocking state via a targeted affirmative selection to safely restore the primary chapter list. This sequence corroborates that robust error correction requires the active anticipation and resolution of intermediate blocking states rather than reliance on blind heuristic fallbacks.

D ANNOTATED SAMPLES

Figures 7, 8, 9, 10, 11, 12, 13, 14, 15, and 16 present ten illustrative audited contract annotations selected to highlight complementary transition, verification, and recovery patterns. The examples span AndroidControl, AITW, and AMEX and cover seven action types: `tap`, `type`, `wait`, `long_press`, `swipe`, `back`, and `home`.

Each example shows the observed state s_t , the annotated action, the resulting state s_{t+1} , and a condensed transition contract. The contract summarizes the grounded task state, action precondition, semantic action, expected state delta, observable appear/disappear evidence (+/-), recovery branches, and verifier hints. Action targets are visualized directly on the interface using a red marker for `tap/long_press`, a red arrow for `swipe`, and a red box for `type`.

The selected examples illustrate several behaviors targeted by TRAVEC: task progress without page navigation, delayed outcomes requiring temporal verification, repeated or overshooting interactions, recovery from unproductive states, and transitions that prepare later stages of a long-horizon task. These cases show how transition contracts associate action semantics with observable state changes and how recovery annotations specify corrective behavior when the realized transition deviates from the intended one.

Text is condensed for presentation, with truncations marked by "...". Complete annotation records will be released in JSON format upon acceptance.

Red: model prediction Blue: expert action / transition Teal: visible evidence Steps are zero-based

(a) Silent transition failure

Goal
Save Qutub Minar to "Delhi Wonder" in the shared category.

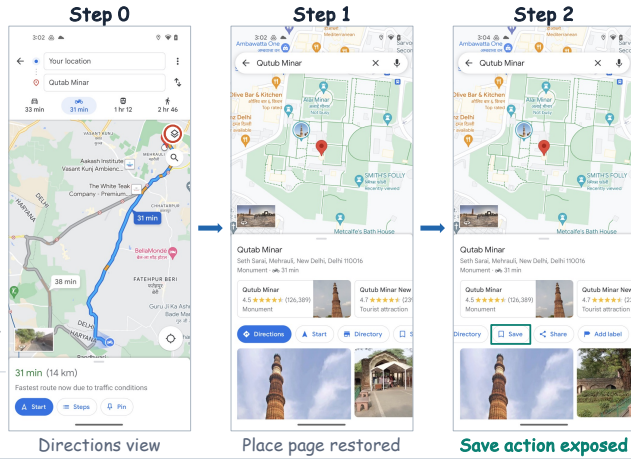
Model prediction · Step 0
Tap map layers

Targets map display controls.
The reference first leaves the directions view.

Expert trajectory
Back → swipe left

Back restores the place page.
A left swipe exposes Save.

Valid syntax can still miss the intended transition.



(b) State drift

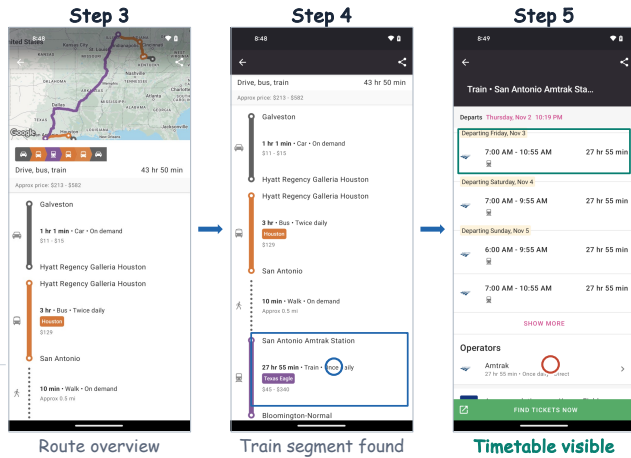
Goal
View the train schedule for the selected route.

Model prediction · Step 5
Tap the Amtrak operator
Would keep interacting after the requested schedule is already visible.

Expert trajectory
Scroll down → tap train.
Then terminate (success)

Opens the train timetable.
At Step 5, the expert stops.

Goal-state tracking must include when to stop.



(c) Recovery

Goal
View algebra chapter details in the math app.

Model prediction · Step 0
System Back

Differs from the reference in-app Back action; its outcome is not observed.

Expert trajectory
In-app Back → Yes

Back opens the quit dialog.
Yes dismisses it and returns to the chapter list.

Clear the blocking dialog to resume task navigation.

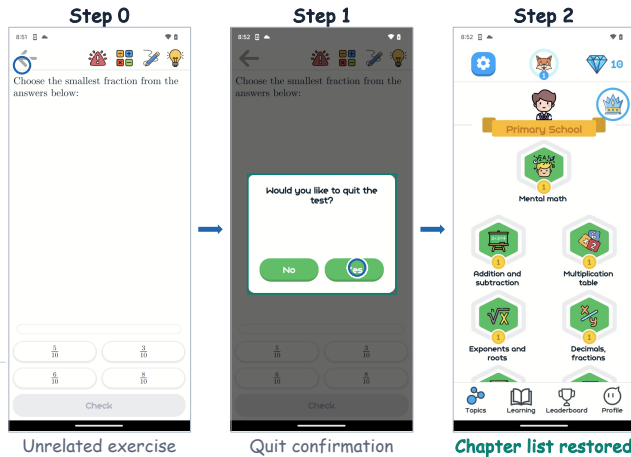
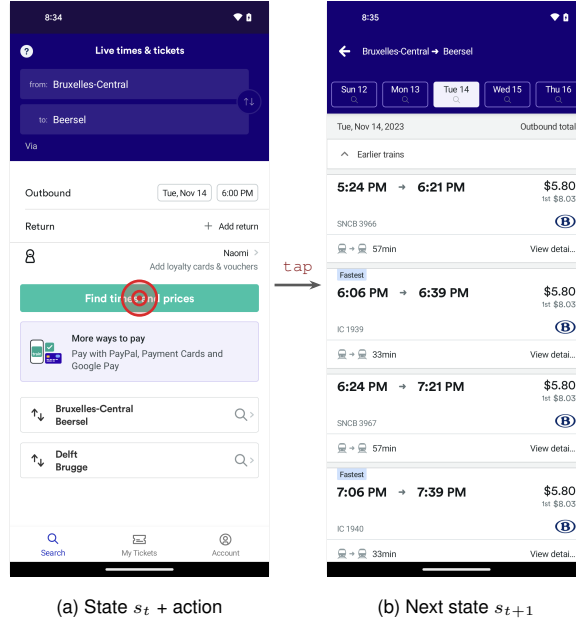


Figure 6: Counterfactual static case studies contrasting unexecuted GUI-agent predictions against ground-truth expert trajectories. Red annotations denote model predictions, blue indicates expert actions, and teal highlights visible evidence.

GoalAndroidControl episode 12896 / step 13 **tap**

"I have to go to Brussels-Central for work-related reasons. Use the trainline app to locate a rail leaving after 6:00 PM on November 14 from Brussels-Central to Beersel."



(c) Generated contract annotation

MODEL STATE

page: 'Live times & tickets' (search_form) app: Trainline
 summary: Trainline journey search form. from: Bruxelles-Central, to: Beersel. Outbound date 'Tue, Nov 14' at '6:00 PM'. A large green primary button labeled 'Find times and prices' is centered on screen, followed by a 'More ways to pay' card ...
 key.elements: 'Find times and prices' primary button (OCR node 18 ...; from: Bruxelles-Central; to: Beersel)

PRECONDITION

✓ satisfied conf. 0.95
 The 'Find times and prices' button must be visible and enabled, with origin/destination/date populated.
 checks: 'Find times and prices' button visible near screen center; from field shows 'Bruxelles-Central'; to field shows 'Beersel'

SEMANTIC ACTION

type: **tap** point: (0.500, 0.516)
 target: 'Find times and prices' primary green button
 grounding: UI.tree.t node id 18, bbox [0.038889,0.491667,0.961111,0.54]

EXPECTED DELTA

page.changed = true
 Tapping the button submits the search and navigates from the journey-search form to the outbound results/timetable page for Bruxelles-Central → Beersel on Tue, Nov 14, 2023.
 +Earlier trains +Outbound total +Tue, Nov 14, 2023
 -Find times and prices -Live times & tickets

RECOVERY BRANCHES

- After tap, screen still shows the search form ('Find times ... ⇒ tap (0.500, 0.516)
- A blank or loading page appears with no train rows for a ... ⇒ wait 2 s
- A date picker, time picker, or station editor opened instead ... ⇒ back

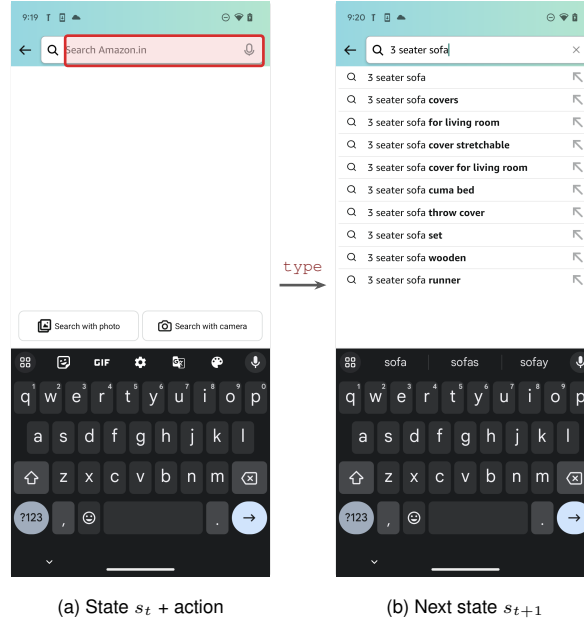
VERIFIER HINTS

progress = advances loop risk = low
 evidence: +text.appeared: Earlier trains +text.appeared: Outbound total +text.appeared: Tue, Nov 14, 2023
 expected.page.next: list.view (train timetable / search results)

Figure 7: (AndroidControl, tap). Screenshots and the generated contract annotation (condensed); item_id: bce40caad880af5cca5c5c40.

Goal

AndroidControl episode 14311 / step 2 type

"I want to find a 3 seater sofa set to give to my friend Abdul Karim on his birthday on the 21st of december."**(c) Generated contract annotation****MODEL STATE**

page.type: search.screen.with.keyboard app: Amazon
 summary: Amazon in-app search entry screen. The top search input shows placeholder 'Search Amazon.in' and is focused; the soft keyboard is open; 'Search with photo' and 'Search with camera' options are shown above the keyboard. No results or ...

PRECONDITION

A focused, editable search field with the soft keyboard already open is the standard precondition for a type action. ✓ satisfied

checks: search field focused; field editable; keyboard visible

SEMANTIC ACTION

type: type text: '3 seater sofa'
 target: Amazon search input field (placeholder 'Search Amazon.in') at top of screen
 grounding: bbox [0.21, 0.06, 0.97, 0.11]

EXPECTED DELTA

page.changed = true

The typed text '3 seater sofa' appears in the search field, the placeholder is replaced, a 'Clear search keywords' (X) button appears, and an autocomplete suggestion list renders below the ...

evidence: text.appeared: 3 seater sofa; text.appeared: 3 seater sofa covers

RECOVERY BRANCHES

- After typing, the search field still shows placeholder ... $\Rightarrow$ tap (0.500, 0.085)
- The field shows partial or altered text (not exactly '3 ... $\Rightarrow$ tap (0.930, 0.085)
- Keyboard is not visible, so typing cannot proceed $\Rightarrow$ tap (0.500, 0.085)

VERIFIER HINTS

loop risk = low no.change.risk = low

evidence: +text '3 seater sofa' visible in search field +autocomplete suggestion list contains '3 seater ... + 'Clear search keywords' button present
 -search field still empty/placeholder

notes: press.enter is false; results/product page has NOT loaded.
 Verifier must not accept 'search results appeared' as ...

Figure 8: (AndroidControl, type). Screenshots and the generated contract annotation (condensed); item_id: 2cf51485dd9b973e83c97087.



Figure 9: (AndroidControl, wait). Screenshots and the generated contract annotation (condensed); item_id: a4e5c920a70850d46d3a4ebb.

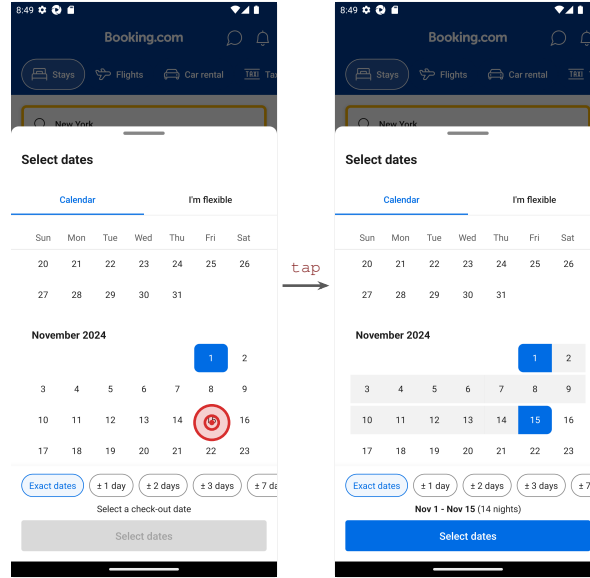


Figure 10: (AndroidControl, long_press). Screenshots and the generated contract annotation (condensed); item_id: 5165d1e151752cac4a1e8c38.

Goal

AMEX episode c7882b1a075f.. / step 9 tap

"Open Booking.com. Search hotels in New York, from 1 November to 15 November. 2 guests and 2 children in 2 rooms with 2 single beds. Filters include balcony. Accept online payments. Sort by distance to downtown."

(a) State s_t + action(b) Next state s_{t+1}

(c) Generated contract annotation

MODEL STATE

page.type: date_picker_bottom_sheet app: com.booking
 summary: Booking.com 'Select dates' calendar bottom sheet is open over a dimmed Stays search page. The 'Calendar' tab is active; November 2024 grid is visible. Nov 1 is highlighted blue as the selected check-in date and the prompt 'Select a ...'
 key.elements: 'Select dates' sheet title; Calendar / I'm flexible tabs; November 2024 month grid

PRECONDITION

checks: date-picker calendar sheet is open; check-in date Nov 1 already selected (blue); 'Select a check-out date' prompt indicates check-out is ...

SEMANTIC ACTION

type: tap point: (0.754, 0.731) 
 target: November 15 date cell in the calendar grid
 intent: Tap the day '15' to set the check-out date, completing the Nov 1 - Nov 15 range.

EXPECTED DELTA

page.changed = false
 True
 +Nov 1 - Nov 15 +14 nights
 -Select a check-out date

RECOVERY BRANCHES

- After tap, range summary does not show 'Nov 1 - Nov 15' and ... $\Rightarrow$ tap (0.754, 0.731)
- A different day range appears (e.g. Nov 1 - Nov 14 or Nov 1 - ... $\Rightarrow$ tap (0.754, 0.731)
- 'Select dates' button remains greyed after selection $\Rightarrow$ tap (0.192, 0.396)

VERIFIER HINTS

progress = positive loop risk = low
 evidence: + 'Nov 1 - Nov 15' text + '14 nights' text
 + enabled blue 'Select dates' button
 notes: Change is a within-sheet content/state update (range completion), not a full navigation; actual.delta labels ...

Figure 11: (AMEX, tap). Screenshots and the generated contract annotation (condensed); item.id: 0e29249d359784689768be20.

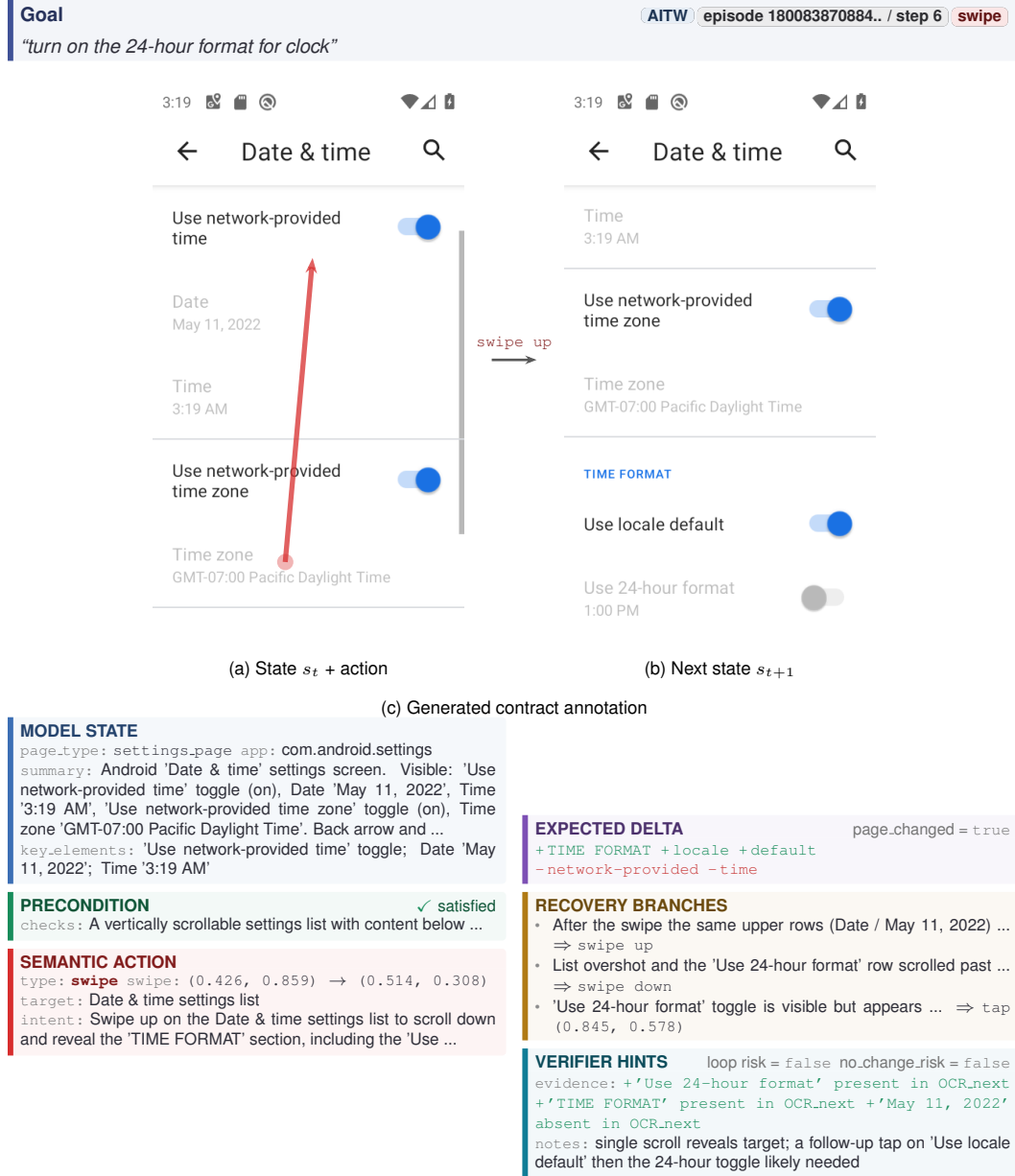


Figure 12: (AITW, swipe). Screenshots and the generated contract annotation (condensed); item_id: b94aec170aeb293c4b832cc2.



Figure 13: (AMEX, type). Screenshots and the generated contract annotation (condensed); item_id: 5103471f383e2e6370f65fe2.

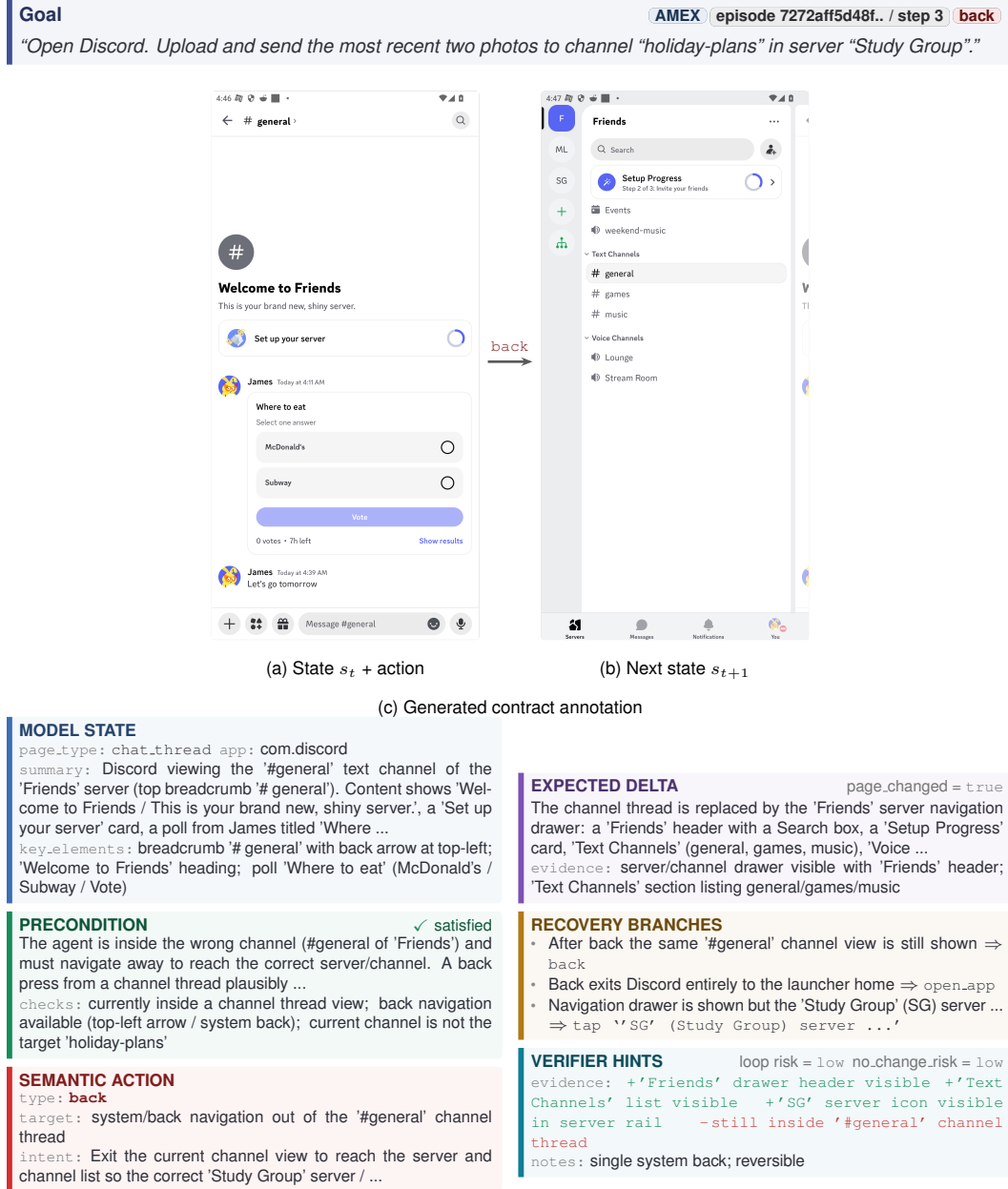
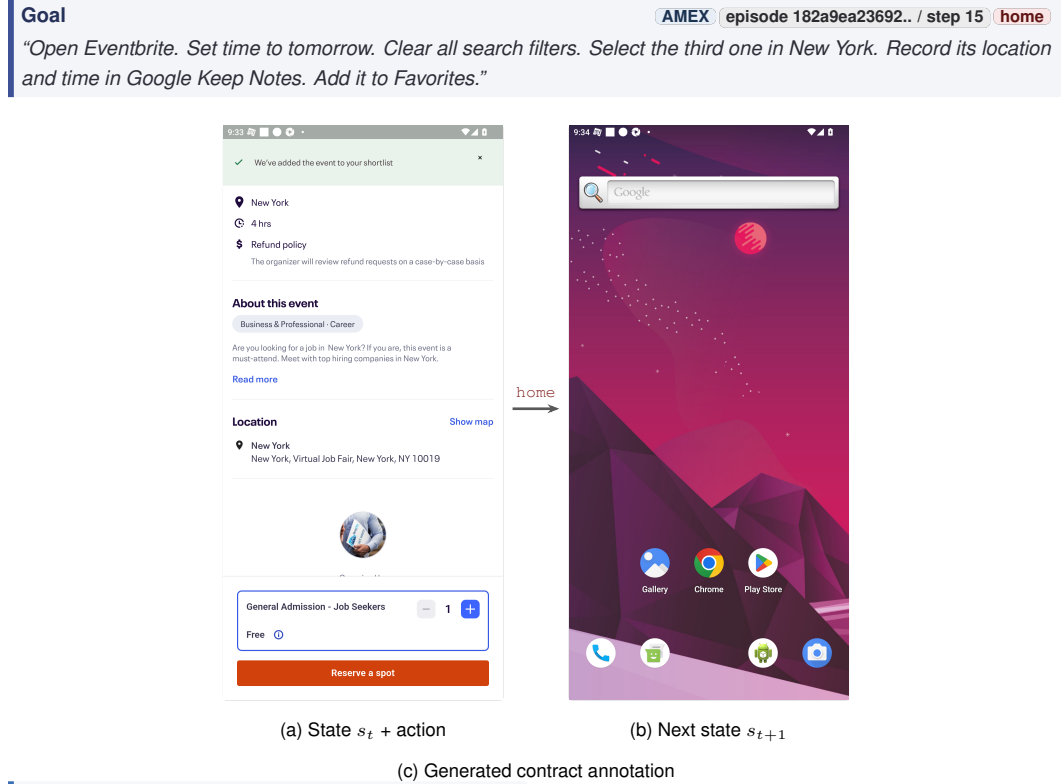


Figure 14: (AMEX, back). Screenshots and the generated contract annotation (condensed); item.id: 6e5f315f466979c0bcef9840.



MODEL STATE page.type: product_detail app: com.eventbrite.attendee summary: Eventbrite event detail page for a New York job-fair event. A green success toast at top reads 'We've added the event to your shortlist'. Below are event facts (location 'New York', duration '4 hrs', refund policy), an 'About this ... key_elements: shortlist-added toast; location: New York; duration: 4 hrs	EXPECTED DELTA page.changed = true App foreground changes from the Eventbrite event detail page to the Android launcher home screen. +launcher home screen +Google search widget +app icons: Gallery, Chrome, Play Store -Eventbrite event detail content -shortlist toast
PRECONDITION ✓ satisfied PRESS_HOME is a global system action that is always available. It is meaningful here because the favorites/shortlist subgoal is already reflected on screen and the next ... checks: Eventbrite event detail page is foreground; system navigation/home affordance is always available; no modal dialog blocks navigation	RECOVERY BRANCHES - After pressing home the Eventbrite page is still foreground ... ⇒ home - Launcher appears but the agent needs Google Keep, which is ... ⇒ open_app - A different app (e.g., recent app) is shown instead of the ... ⇒ home
SEMANTIC ACTION type: home target: system home button intent: Leave the Eventbrite event detail page and return to the launcher home screen, a preparatory navigation step toward ...	VERIFIER HINTS progress = advances by ... loop risk = low evidence: +screenshot_next is launcher home (not Eventbrite) +page.changed true in actual_delta +image_similarity 0.316 confirms large visual change

Figure 15: (AMEX, home). Screenshots and the generated contract annotation (condensed); item_id: 5e5ca51a438767a72724dc96.



Figure 16: (AMEX, swipe). Screenshots and the generated contract annotation (condensed); item_id: 974a3a3019b1f3da39adcdac.